

Hoja de Trabajo 03: OpenMPI comunicación entre procesos

I. Objetivo

Continuar ejercitando conceptos con Open MPI de forma local e identificar el uso de funciones fundamentales para reconocer procesos, realizar comunicación punto a punto, difundir información y distribuir datos entre varios procesos.

II. Instrucciones

- Entregar los archivos de código modificado, correspondientes a los tres ejercicios, debidamente identificado.
- Entregar un documento en formato PDF que contenga las respuestas a las preguntas de análisis de los cuatro ejercicios, e incluir capturas de pantalla como evidencia de la compilación y ejecución correcta de cada programa. Las capturas deben permitir observar claramente los comandos utilizados y los resultados obtenidos.
- Verificar antes de la entrega que todos los programas compilen y se ejecuten correctamente con la cantidad de procesos indicada en cada ejercicio.

III. Ejercicios

En MPI, todos los procesos ejecutan el mismo programa, pero cada proceso posee un identificador único llamado rank, el cual permite que un mismo programa asigne tareas o comportamientos diferentes a cada proceso.

Una empresa posee una oficina central y varias sucursales. Para simular la operación distribuida de la empresa, cada proceso MPI representará una oficina o sucursal diferente. Se utilizará la siguiente equivalencia entre los procesos MPI y las ubicaciones de la empresa:

RANK	REPRESENTACIÓN/SUCURSAL
rank 0	Oficina central
rank 1	Sucursal 1
rank 2	Sucursal 2
rank 3	Sucursal 3
...	

Cada proceso ejecutará el mismo programa, pero utilizará su rank para identificar qué ubicación representa. En esta primera actividad no existirá todavía comunicación entre los procesos. El objetivo es reconocer cómo funciona el entorno MPI y comprobar que cada proceso posee un identificador propio.

Ejercicio 1. Ejecuta el programa HT3_inciso1.c utilizando diferentes 2, 4 y 6 procesos:

Compilación: `mpicc HT3_inciso1.c -o HT3_inciso1`

Ejecución: `mpirun -np x ./HT3_inciso1`

Donde **x** representa la cantidad de procesos que se desean ejecutar.

a. Preguntas de análisis

- ¿Qué representa MPI_COMM_WORLD dentro de un programa MPI?
- ¿Qué ventaja proporciona usar MPI_COMM_WORLD como comunicador inicial en lugar de manejar manualmente una lista de procesos?
- En una ejecución local sobre una sola computadora, ¿por qué los procesos MPI siguen considerándose independientes aunque físicamente utilicen la misma máquina?

Ejercicio 2. Comunicación directa entre dos procesos:

La oficina central necesita recibir el reporte diario de ventas de una de sus sucursales. La sucursal 1 (rank 1) calculará o definirá el total de ventas del día y enviará este valor directamente a la oficina central (rank 0).

La comunicación se realizará utilizando MPI_Send() y MPI_Recv().

MPI_Send()	Estructura de MPI_Recv()
MPI_Send(buffer, count, datatype, destination, tag, communicator);	MPI_Recv(buffer, count, datatype, source, tag, communicator, status);

a. Identifica dentro de las instrucciones MPI_Send() y MPI_Recv(), utilizadas en el programa HT3_inciso2.c, qué valor fue asignado a cada uno de sus parámetros.

Explica brevemente qué representa cada uno de esos valores dentro de la simulación de la empresa. Completa la siguiente tabla a partir de estas llamadas:

MPI_Send()		MPI_Recv()	
Parámetro	Valor utilizado en el programa	Parámetro	Valor utilizado en el programa
buffer		buffer	
count		count	
datatype		datatype	
destination		source	
tag		tag	
communicator		communicator	
		status	

b. Modifica el programa HT3_inciso2.c para que la Sucursal 1 envíe a la Oficina Central un segundo dato: la cantidad de pedidos procesados durante el día.

- Utiliza una nueva variable de tipo entero, por ejemplo: int pedidos = 48;
- La Sucursal 1 deberá enviar: ventas del día y cantidad de pedidos procesados.
- La Oficina Central deberá recibir ambos valores y mostrar en pantalla un resultado similar a:
Ventas reportadas por Sucursal 1: Q1250.75
Pedidos procesados por Sucursal 1: 48

c. Preguntas de análisis

- ¿Por qué se utilizaron tag diferentes para el envío de ventas y pedidos?
- ¿Qué ocurriría si ambos mensajes fueran enviados con el mismo tag y la Oficina Central realizara dos recepciones consecutivas?
- ¿Qué relación debe existir entre los parámetros utilizados en MPI_Send() y los parámetros correspondientes de MPI_Recv() para que la comunicación sea correcta?
- Si la Oficina Central cambiara el source del segundo MPI_Recv() a un proceso distinto de rank = 1, ¿qué efecto tendría sobre la ejecución?
- En este ejercicio se utilizan dos llamadas a MPI_Send() y dos a MPI_Recv(). ¿Qué ventaja tiene mantener ambos datos como mensajes separados en lugar de enviarlos como un único dato?

vi.

Ejercicio 3. Difusión de información a todos los procesos:

La Oficina Central necesita comunicar un cambio de precio a todas las sucursales de la empresa. El nuevo precio será definido únicamente por la Oficina Central (rank 0) y deberá ser recibido por todos los demás procesos que representan las sucursales.

La comunicación se realizará utilizando MPI_Bcast(). La estructura general de la instrucción es:

MPI_Bcast ()
MPI_Bcast(buffer, count, datatype, root, communicator);

- a. Identifica dentro de la instrucción MPI_Bcast() utilizada en el programa HT3_inciso3.c qué valor fue asignado a cada uno de sus parámetros.

Explica brevemente qué representa cada uno de esos valores dentro de la simulación de la empresa. Completa la siguiente tabla a partir de estas llamadas:

MPI_Send()	
Parámetro	Valor utilizado en el programa
buffer	
count	
datatype	
root	
communicator	

- b. Modifica el programa HT3_inciso3.c para que la Oficina Central comunique a todas las sucursales, además del nuevo precio, un porcentaje de descuento que deberá aplicarse durante el día.

- Utiliza una nueva variable, por ejemplo float descuento = 10.0;
- La Oficina Central deberá definir y distribuir ambos datos: Nuevo precio: Q25.50 y Descuento: 10 %
- Utiliza una llamada independiente a MPI_Bcast() para cada dato.
- Cada sucursal deberá mostrar (por ejemplo):
 Sucursal 1: precio recibido = Q25.50
 Sucursal 1: descuento recibido = 10.00 %

c. Preguntas de análisis

- ¿Qué función cumple el parámetro root dentro de MPI_Bcast()?
- ¿Por qué solamente el proceso rank 0 necesita asignar inicialmente un valor a la variable precio?
- ¿Qué diferencia existe entre utilizar MPI_Bcast() para distribuir el precio y realizar múltiples llamadas a MPI_Send() desde la Oficina Central?
- ¿Qué ocurriría si en una de las sucursales se especificara un valor de root diferente al utilizado por los demás procesos en la misma llamada a MPI_Bcast()?
- ¿Por qué todos los procesos que pertenecen a MPI_COMM_WORLD deben participar en la ejecución de MPI_Bcast()?
- Después de ejecutar MPI_Bcast(), ¿qué valor contiene la variable precio en cada uno de los procesos?

vii.

Ejercicio 4. Distribución del trabajo:

La Oficina Central dispone de una lista de pedidos que deben ser procesados por las distintas sucursales. En lugar de enviar manualmente un conjunto de pedidos a cada proceso, se utilizará `MPI_Scatter()` para distribuir partes del conjunto de datos entre todos los procesos.

La comunicación se realizará utilizando `MPI_Scatter()`. La estructura general de la instrucción es:

MPI_Scatter()
<pre>MPI_Scatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, communicator);</pre>

- a. Identifica dentro de la instrucción `MPI_Scatter()` utilizada en el programa `HT3_inciso4.c` qué valor fue asignado a cada uno de sus parámetros.

Completa la siguiente tabla a partir de estas llamadas. Explica brevemente qué representa cada uno de esos valores dentro de la simulación de la empresa.:

MPI_Scatter()	
Parámetro	Valor utilizado en el programa
<code>sendbuf</code>	
<code>sendcount</code>	
<code>sendtype</code>	
<code>recvbuf</code>	
<code>recvcount</code>	
<code>recvtype</code>	
<code>root</code>	
<code>communicator</code>	

- b. Modifica el programa `HT3_inciso4.c` para que la Oficina Central distribuya dos datos a cada ubicación en lugar de uno.

- Cada ubicación deberá recibir: cantidad de pedidos y cantidad de empleados disponibles para procesarlos.
- Para realizar la distribución, modifica los parámetros `sendcount` y `recvcount` de `MPI_Scatter()`.
- Cada proceso deberá mostrar los datos recibidos.

c. Preguntas de análisis

- ¿Qué diferencia existe entre `sendbuf` y `recvbuf` dentro de `MPI_Scatter()`?
- ¿Qué determina el parámetro `sendcount` y cómo afecta la cantidad de datos que recibe cada proceso?
- En el programa base, ¿qué parte del arreglo `pedidos` recibe cada proceso y qué relación tiene esto con su `rank`?
- ¿Por qué solamente el proceso definido como `root` necesita contener inicialmente todos los datos que serán distribuidos?
- Si el arreglo del proceso `root` contiene menos datos de los necesarios para la cantidad de procesos y el valor de `sendcount`, ¿qué problema podría producirse?